# Exercise 2: Give Claude Your Team's Decisions

**Session**: 2 — Steering Claude
**Duration**: 34 minutes, plus a 4-minute closing round
**Project**: The same BookStore API.

## Goal

Your team made three decisions that the code does not show. The prompt for the task leaves more decisions open. In this exercise you:

1. Check a prompt, and list what it leaves to Claude.
2. Run the task, and save what Claude builds.
3. Write your team's decisions where Claude reads them.
4. Run the same task again, and compare the two results.

## Start here

- Open a terminal in the `bookstore` folder that you cloned.
- Training mode is on: Claude asks you questions before it helps. Stuck? Type *just tell me* in the chat.
- Never edit `CLAUDE.md`.
- Write your notes on paper or in a text file outside the project.
- The trainer tells you when to start each next task.

## Your team's decisions

Your team agreed on these three decisions in a meeting. They are not in the repo, so Claude cannot read them.

1. **Test kit.** "We built a test kit: `src/test/kotlin/bookstore/testkit/TestKit.kt`. New handler tests use it. We do not rewrite the old tests."
2. **Bad input.** "Bad input gives a 400, not a 500. New endpoints do this. We fix the old endpoints later, in a separate change."
3. **Small changes.** "A change touches only the files the task needs. No extra changes."

The old tests and the old handlers still show the old way.

# Tasks

## 1. Check the prompt before you run it

This task produces a list in your notes: the decisions that one short prompt leaves to Claude. You run this prompt in task 2.

1. **Read the prompt.** Do not run it yet. Task 2 explains the part in brackets:

```
[Exercise 2 experiment — execute directly, no leading questions.] Add a DELETE
/api/reviews/{id} endpoint to the BookStore API, with tests.
```

2. **List what it leaves open.** Write down every decision Claude must make because the prompt does not say it. Think of the questions a reviewer asks about the result. Two examples from another task, an "edit your profile" endpoint: *Which fields may a user change? What does the user get back?*

3. **Mark each decision.** Write *task* when it is about this endpoint only. Write *team* when your team decides it once, for every endpoint.

4. **Write what you would add.** For the *task* lines, write the two or three sentences you would add to the prompt. Use CONTEXT-TASK-OUTCOME. End with a *Define done* sentence: what must be true when Claude is done.

5. **Check your sentences (optional).** Start `claude` and run `/prompt-coach 4`. As your next message, paste your list and your sentences. The coach gives one round of feedback. Then type `/exit`.

6. **Predict.** Write *yes* or *no* for each of your team's three decisions: will Claude follow it without being told? A wrong prediction is fine.

Why you do not run your own version: the prompt stays the same in task 2 and task 4. Then the two results show only what your rule files change.

**Done when**: your list has at least four decisions, each marked *task* or *team*, and your sentences and three predictions are written down.

## 2. Run the task without the decisions

This task produces `session2-before.diff`: a file with every change Claude makes when it knows only the code.

1. **Get a clean start.** Run `./gradlew test` and `git status`. A test fails, or `git status` lists changes? Start `claude`. Press Shift+Tab until the line under the chat box says auto mode: Claude then does not ask permission. Run `/catch-up 1`. It finishes the session 1 code work and commits it. Step 4 removes work that is not committed. Both clean? Go to step 2.
2. **Start a fresh session.** Type `/exit` if Claude is open, then start `claude`. A fresh session always means this: close Claude and start it again, never `/clear`. Paste the prompt from task 1, with the brackets. The tag at the start switches training mode off for this one prompt, so Claude builds instead of asking questions.
3. **Run the tests.** When Claude is done, run `./gradlew test`. Write down whether all tests pass.
4. **Save the result.** Run `/save-changes before`. It saves Claude's changes in `session2-before.diff` and removes them from the project. Then type `/exit`.

**Done when**: `session2-before.diff` exists.

## 3. Put the decisions where Claude reads them

This task produces `.claude/rules/testing.md`, `.claude/rules/handlers.md` and `CLAUDE.local.md`. Decisions 1 and 2 go in rule files with `paths:`. Decision 3 goes in `CLAUDE.local.md` (*Rule Discovery: With or Without Paths*).

1. **Decision 1.** Start `claude` and paste this block. Claude creates the rule file:

```
[Exercise 2 experiment — execute directly, no leading questions.] Create
.claude/rules/testing.md with exactly this content, nothing added or changed:


---
description: How we write new handler tests
paths:
  - "src/test/kotlin/bookstore/handler/**/*.kt"
---
New handler tests use the test kit in src/test/kotlin/bookstore/testkit/TestKit.kt.
Read that file before you write a test. Use this shape: one <Method><Resource>ApiTest
class per endpoint, every test wrapped in apiTest { }.
A new test never uses TestEnv().use or env.request(), even when the tests around it do.
Never convert an existing test to the kit, also not in a file you edit.
```

2. **Decision 2.** In your editor, create `.claude/rules/handlers.md`. Copy the shape of `testing.md`, with `"src/main/kotlin/bookstore/handler/**/*.kt"` in `paths:`. Write two sentences. First: *New endpoints answer bad input, such as an id that is*

*not a number, with 400.* Second: what happens to the old endpoints for now. They are in the `src/main/kotlin/bookstore/handler/` folder. A rule is checkable when a reader of a diff can say "this breaks the rule".

3. **Decision 3.** In your editor, create `CLAUDE.local.md` in the `bookstore` folder, or add to it if it exists. Write two lines. First, copy this line exactly: "*Before you write or edit a file under src/test/kotlin/bookstore/handler/ or src/main/kotlin/bookstore/handler/, read the matching file in .claude/rules/.*" Second, decision 3 as a checkable rule: name what a change may touch. An example about another subject: "*A bug fix may touch only the file with the bug and its test.*"

Why the first line: a `paths:` rule loads when Claude opens a matching file. A new file, or an edit through a script, may not load it. Claude reads `CLAUDE.local.md` at the start of every session.

**Done when**: the three files exist, `handlers.md` has a `paths:` line and says what happens to the old endpoints, and `git status` does not list `CLAUDE.md`.

## 4. Run the same task with the decisions

This task produces `session2-after.diff`.

1. **Start a fresh session.** Type `/exit` and start `claude` again. Claude now reads your new files at the start.
2. **Paste the prompt from task 1 again**, tag included. In the chat, watch which files Claude reads before it writes a test. Write down whether it reads the test kit.
3. **Run the tests.** When Claude is done, run `./gradlew test`. Write down whether all tests pass.
4. **Save the result.** Run `/save-changes after`. Then type `/exit`.

**Done when**: `session2-after.diff` exists.

## 5. Closing: compare the two runs

This task gives you a report on each decision, with the lines that decided it.

1. **Run the comparison.** Start `claude` and run `/verify-exercise 2`. It compares your two diffs with the three decisions. First it asks which decision Claude still broke in the second run. Answer in one line, from what you saw in the chat. *None* is a fine answer.
2. **Read the report.** Each decision is *followed*, *broken* or *not tested*. *Not tested* means the task gave Claude no chance to break it.

3. **Write down four lines.** Line 1: your predictions, next to what the first run did. Line 2: the decision that changed most between the two runs. Line 3: one sentence from your rules that you would write differently, or the one that helped most. Line 4: one decision from your task 1 list that Claude made by itself, and what it chose.

**Done when**: four lines are written down. Bring them to the closing round.

## Bonus (only if time remains)

**Check your rules, then try a new task.** Run `/context-coach 2`. As your next message, paste your `handlers.md` and your decision 3 line. Improve the one thing it names. Then start a fresh session and paste:

```
[Exercise 2 experiment — execute directly, no leading questions.] Add a GET
/api/authors/{id}/books endpoint to the BookStore API, with tests.
```

Run `/save-changes bonus`, then `/verify-exercise 2` in a fresh session.

**Change an old test.** Start a fresh session and paste:

```
[Exercise 2 experiment — execute directly, no leading questions.] In the existing review
tests, make the test for listing reviews also check that the first review has rating 5.
```

Run `git diff`. Did Claude change only that test, or did it convert it to the kit? Then run `/save-changes probe` to remove the changes.

## Back at work: try this on your own project

Pick one review comment that you write often. Write it as a rule with a `paths:` line. In a real team, `.claude/rules/` is committed, so every teammate gets the same rules. In this course, git ignores it.

## Appendix: `/save-changes` by hand

If the command stops, it says why. Or run these from the project folder, one per line. The first two save your changes. The last three remove them:

```
git add -A .
git diff --cached --output=session2-before.diff
git reset -q .
git checkout -- .
git clean -fd .
```

Replace `before` with `after`, `bonus` or `probe`. The last command deletes new files that git does not track yet. Your rule files, `CLAUDE.local.md` and the saved diffs stay: `git clean` skips files that git ignores.

## Closing round

The trainer asks the room. Have these answers ready:

- Which decision did Claude follow without being told? Was your prediction right?
- Which line from your task 1 list belongs in the prompt, and which belongs in a rule? Why?
- Why does decision 1 go in a rule file with `paths:`, and decision 3 in `CLAUDE.local.md`? Why does `CLAUDE.local.md` also point to the rules?
- Which words in your rule for decision 2 make it checkable?